

OzoneX

smart contracts final audit report

October 2025



hashex.org



contact@hashex.org

Contents

1. Disclaimer	3
2. Overview	4
3. Project centralization risks	6
4. Found issues	7
5. Contracts	8
6. Conclusion	13
Appendix A. Issues' severity classification	14
Appendix B. Issue status description	15
Appendix C. List of examined issue types	16
Appendix D. Centralization risks classification	17

1. Disclaimer

This is a limited report on our findings based on our analysis, in accordance with good industry practice at the date of this report, in relation to cybersecurity vulnerabilities and issues in the framework and algorithms based on smart contracts, the details of which are set out in this report. In order to get a full view of our analysis, it is crucial for you to read the full report. While we have done our best in conducting our analysis and producing this report, it is important to note that you should not rely on this report and cannot claim against us on the basis of what it says or doesn't say, or how we produced it, and it is important for you to conduct your own independent investigations before making any decisions. We go into more detail on this in the disclaimer below - please make sure to read it in full.

By reading this report or any part of it, you agree to the terms of this disclaimer. If you do not agree to the terms, then please immediately cease reading this report, and delete and destroy any and all copies of this report downloaded and/or printed by you. This report is provided for information purposes only and on a non-reliance basis and does not constitute investment advice. No one shall have any right to rely on the report or its contents, and HashEx and its affiliates (including holding companies, shareholders, subsidiaries, employees, directors, officers, and other representatives) (HashEx) owe no duty of care towards you or any other person, nor does HashEx make any warranty or representation to any person on the accuracy or completeness of the report. The report is provided "as is", without any conditions, warranties, or other terms of any kind except as set out in this disclaimer, and HashEx hereby excludes all representations, warranties, conditions, and other terms (including, without limitation, the warranties implied by law of satisfactory quality, fitness for purpose and the use of reasonable care and skill) which, but for this clause, might have effect in relation to the report. Except and only to the extent that it is prohibited by law, HashEx hereby excludes all liability and responsibility, and neither you nor any other person shall have any claim against HashEx, for any amount or kind of loss or damage that may result to you or any other person (including without limitation, any direct, indirect, special, punitive, consequential or pure economic loss or damages, or any loss of income, profits, goodwill, data, contracts, use of money, or business interruption, and whether in delict, tort (including without limitation negligence), contract, breach of statutory duty, misrepresentation (whether innocent or negligent) or otherwise under any claim of any nature whatsoever in any jurisdiction) in any way arising from or connected with this report and the use, inability to use or the results of the use of this report, and any reliance on this report. The analysis of the security is purely based on the smart contracts alone. No applications or operations were reviewed for security. No product code has been reviewed. HashEx owns all copyright rights to the text, images, photographs, and other content provided in the following document. When using or sharing partly or in full, third parties must provide a direct link to the original document mentioning the author (hashex.org).

2. Overview

HashEx was commissioned by the OzoneX team to perform an audit of their smart contract. The audit was conducted between 22/10/2025 and 24/10/2025.

The purpose of this audit was to achieve the following:

- Identify potential security issues with smart contracts
- Formally check the logic behind given smart contracts.

Information in this report should be used for understanding the risk exposure of smart contracts, and as a guide to improving the security posture of smart contracts by remediating the issues that were identified.

The code is available in the BNB Smart Chain mainnet at 0xEfE12Eb89bAe92F57B0E7BA3C4dfEee9627eE12a.

2.1 Summary

Project name	OzoneX
URL	https://ozonex.tech/
Platform	Binance Smart Chain
Language	Solidity
Centralization level	● High
Centralization risk	● High

2.2 Contracts

Name	Address
OZONE	0xEfE12Eb89bAe92F57B0E7BA3C4dfEee9627eE12a

3. Project centralization risks

Ce3CR4f Governance functions

The owner can update the **treasuryWallet** address that collects transfer taxes.

The owner can modify the **isExemptFromTax** and **isCEXAddress** lists, whose members are exempt from the transfer tax.

The owner can modify the **authorizedBurners** list, whose members can increase the **totalDistributedRewards** by arbitrary number.

The owner can update the **monthlyProfitUSDT** and **currentNickelPriceUSDT** variables without sanity limits.

The owner can flip pausing state to allow or deny all transfers at once.

Recommendations

Recommendation

Secure the ownership with a Timelock contract with MultiSig account as admin.

Comments

Ozone team comment

We acknowledge the observation on owner privileges.

The current ownership is held by our internal treasury account, which serves multiple operational purposes, including handling presale proceeds, staking rewards, and Proof-of-Reserves tracking.

4. Found issues



Medium	1 (20%)
Low	2 (40%)
Info	2 (40%)

Ce3. OZONE

ID	Severity	Title	Status
Ce3le4	Medium	Incorrect return data	✓ Acknowledged
Ce3le6	Low	Lack of events	✓ Acknowledged
Ce3le2	Low	Unused code and code without effect	✓ Acknowledged
Ce3le5	Info	Unclear purpose of getProofOfReserves function	✓ Acknowledged
Ce3le3	Info	Transfer tax bypassing	✓ Acknowledged

5. Contracts

Ce3. OZONE

Overview

An [ERC20](#) standard token contract. The initial supply is fixed, i.e. there's no active minting functionality after deployment. The token contract is inherited from OpenZeppelin's ERC20 implementation, which is considered the best practice.

Token transfers are affected by the contract's pausing state. Transfer tax with fixed percentage is applied to ordinary transfers that don't include explicitly whitelisted addresses.

Contract description states that OZONE token represents real-time nickel ore and is backed by physical nickel ore trading and shipments. There are no in-code confirmations of these statements. Users should either request for proof from the project owners or trust them blindly.

Issues

Ce3le4 Incorrect return data

● Medium

✓ Acknowledged

The `getTransferAmount()` function returns invalid data if from or to addresses are not in the `isExemptFromTax` list, but included into `isCEXAddress` or equal to OZONE contract's address.

```
function getTransferAmount(
    uint256 amount,
    address from,
    address to
) external view returns (uint256 transferAmount, uint256 taxAmount) {
    if (isExemptFromTax[from] || isExemptFromTax[to]) {
        return (amount, 0);
    }

    taxAmount = (amount * TAX_RATE) / BASIS_POINTS;
    transferAmount = amount - taxAmount;
```



```
        return (transferAmount, taxAmount);
    }
```

Recommendation

The `getTransferAmount()` function should be marked as unreliable.

Ozone team response

We acknowledge the finding and will mark this function as **informational/unreliable** in documentation to prevent external reliance.

Ce3le6 Lack of events

● Low

👍 Acknowledged

Several administrative functions (e.g., `emergencyWithdraw()`, `addToDistributedRewards()`) do not emit events upon execution. This lack of event emission reduces on-chain transparency, makes it harder for off-chain services or auditors to track important actions, and may hinder incident monitoring or analytics.

Ozone team comment

Acknowledged. Will be implemented in future versions or secondary administrative contracts (e.g., staking/presale modules).

Ce3le2 Unused code and code without effect

● Low

👍 Acknowledged

The `getFixedTotalSupply()` and `getRemainingSupply()` functions return meaningless constants. The `TOTAL_SUPPLY` variable has its own getter.

```
function getFixedTotalSupply() external pure returns (uint256) {
    return TOTAL_SUPPLY;
}
```

```
// Note: This function always returns 0 since total supply is fixed and fully minted
// Kept for interface compatibility but marked as deprecated
```

```
function getRemainingSupply() external pure returns (uint256) {  
    return 0; // All tokens are already minted (fixed supply model)  
}
```

The `isDEXAddress[]` address mapping is not in use. It can be modified, but there's not a single method to read `isDEXAddress` values.

Ozone team comment

Acknowledged. Functions are kept for compatibility purposes; no impact on security.

Ce3le5 Unclear purpose of getProofOfReserves function

● Info

👍 Acknowledged

The `getProofOfReserves()` function returns on-chain data that can be set by the owner to arbitrary values.

```
function getProofOfReserves()  
    external  
    view  
    returns (  
        uint256 treasuryBalance,  
        uint256 monthlyProfit,  
        uint256 distributedRewards,  
        uint256 availableForRewards  
    )  
{  
    return (  
        address(treasuryWallet).balance,  
        monthlyProfitUSDT,  
        totalDistributedRewards,  
        monthlyProfitUSDT > totalDistributedRewards  
            ? monthlyProfitUSDT - totalDistributedRewards  
            : 0  
    );  
}
```

Ozone team comment

The function is not meant to provide cryptographic proof, but to serve as an informational bridge between RWA operations and token metrics.

Ce3le3 Transfer tax bypassing

● Info

✔ Acknowledged

P2P transfers are subjected to transfer tax, but at the same time P2P transfers can be made via tax-exempt CEX hot wallet.

```
function _transferWithTax(
    address from,
    address to,
    uint256 amount
) internal {
    require(from != address(0), "Transfer from zero address");
    require(to != address(0), "Transfer to zero address");

    // No tax scenarios:
    // 1. Exempt addresses (treasury, presale contract, etc)
    // 2. CEX addresses (Binance, Coinbase, etc)
    // 3. Contract itself
    if (
        isExemptFromTax[from] ||
        isExemptFromTax[to] ||
        isCEXAddress[from] ||
        isCEXAddress[to] ||
        from == address(this) ||
        to == address(this)
    ) {
        // No tax for exempt/CEX addresses
        _transfer(from, to, amount);
    } else {
        // Apply 1% tax for:
        // - DEX trading (PancakeSwap, etc)
        // - P2P transfers
        // - Any other non-exempt transfers
        uint256 taxAmount = (amount * TAX_RATE) / BASIS_POINTS;
        uint256 transferAmount = amount - taxAmount;
```

```
        // Transfer tax to treasury
        if (taxAmount > 0) {
            _transfer(from, treasuryWallet, taxAmount);
            emit TaxCollected(from, to, taxAmount);
        }

        // Transfer remaining amount
        _transfer(from, to, transferAmount);
    }
}
```

Ozone team comment

Expected behavior. We will clarify this in public documentation and token policy.

6. Conclusion

1 medium, 2 low severity issues were found during the audit. No issues were resolved in the update. The reviewed contracts are highly dependent on the owner's account. See the centralization risks chapter. This audit includes recommendations on code improvement and the prevention of potential attacks.

Appendix A. Issues' severity classification

- **Critical.** Issues that may cause an unlimited loss of funds or entirely break the contract workflow. Malicious code (including malicious modification of libraries) is also treated as a critical severity issue. These issues must be fixed before deployments or fixed in already running projects as soon as possible.
- **High.** Issues that may lead to a limited loss of funds, break interaction with users, or other contracts under specific conditions. Also, issues in a smart contract, that allow a privileged account the ability to steal or block other users' funds.
- **Medium.** Issues that do not lead to a loss of funds directly, but break the contract logic. May lead to failures in contracts operation.
- **Low.** Issues that are of a non-optimal code character, for instance, gas optimization tips, unused variables, errors in messages.
- **Informational.** Issues that do not impact the contract operation. Usually, informational severity issues are related to code best practices, e.g. style guide.

Appendix B. Issue status description

- ✔ **Resolved.** The issue has been completely fixed.
- 🔄 **Partially fixed.** Parts of the issue have been fixed but the issue is not completely resolved.
- 🕒 **Acknowledged.** The team has been notified of the issue, no action has been taken.
- ❓ **Open.** The issue remains unresolved.

Appendix C. List of examined issue types

- Business logic overview
- Functionality checks
- Following best practices
- Access control and authorization
- Reentrancy attacks
- Front-run attacks
- DoS with (unexpected) revert
- DoS with block gas limit
- Transaction-ordering dependence
- ERC/BEP and other standards violation
- Unchecked math
- Implicit visibility levels
- Excessive gas usage
- Timestamp dependence
- Forcibly sending ether to a contract
- Weak sources of randomness
- Shadowing state variables
- Usage of deprecated code

Appendix D. Centralization risks classification

Centralization level

- **High.** The project owners can manipulate user's funds, lock user's funds on their will (reversible or irreversible), or maliciously update contracts parameters or bytecode.
- **Medium.** The project owners can modify contract's parameters to break some functions of the project contract or contracts, but user's funds remain withdrawable.
- **Low.** The contract is trustless or its governance functions are safe against a malicious owner.

Centralization risk

- **High.** Lost ownership over the project contract or contracts may result in user's losses. Contract's ownership belongs to EOA or EOAs, and their security model is unknown or out of scope.
- **Medium.** Contract's ownership is transferred to a contract with not industry-accepted parameters, or to a contract without an audit. Also includes EOA with a documented security model, which is out of scope.
- **Low.** Contract's ownership is transferred to a well-known or audited contract with industry-accepted parameters.

 contact@hashex.org

 [@hashex_manager](https://t.me/hashex_manager)

 blog.hashex.org

 [linkedin](https://www.linkedin.com/company/hashex)

 [github](https://github.com/hashex)

 [twitter](https://twitter.com/hashex)

#HashEx
BLOCKCHAIN SECURITY